Paratemporal Methods for Reinforcement Learning

Plan and progress (RL track)

ParaDEVS Library workspace, Binghamton

Started October 10, 2026

Abstract

This plan identifies where the paratemporal engine (exact or sampled branching with merging) can give reinforcement learning something standard methods do not. It analyses the variance consequences of Monte Carlo sampling at every stage combined with merging, proposes toy problems with known answers and standard Python baselines, and surveys how the Java/MS4 engine and a Python learner should communicate. A progress section records what has been built (Section 8).

ParaDEVS documents. The entry point is **Start Here**, a reading guide that links every document.

- **Build Report**: what has been built, how it was verified, what is next
- **How-To Guide**: setup, tutorials, API reference, troubleshooting
- **Architecture Guide**: how the library works and how to read results
- **RL Plan** (this document): reinforcement-learning plan, analysis and results
- **Control Spec**: action selection: Para.decide, graph-mode planning, value and policy iteration
- **Hybrid Study**: which draws to enumerate or sample, and how cost grows with lookahead
- **Fast-Copy Proposal**: why the engine replays steps, where time goes, how to speed it up

Contents

1 Where we are today **2**

2 Where paratemporal fits in RL **2**

3 Monte Carlo at every stage, plus merging: variance **3**

4 Toy problems **4**

5 Python baselines **4**

6 Connecting the Java/MS4 engine to Python **5**

7 Phased plan **6**

8 Progress **6**

8.1 R0: environment (done 2026-10-10) 6

8.2 R1: Python reference prototypes (in progress) 7

8.3 T1: 19-state random walk, n-step TD prediction 7

8.4 T2: slippery FrozenLake 8×8, prediction 8

8.5 What R1 shows 8

8.6 R1 in the Java engine: n-step targets with k branches (2026-10-14) 9

1 Where we are today

- The Demo Deliverable's RL side is `ParaDEVS_Barb_Q_Constructor.ipynb` (Colab, PyTorch). It runs tabular Q-learning on stock CSVs, then **row-normalises the Q-table into action probabilities** and saves it as a CSV. The Java engine reads that CSV as a stochastic policy (the `Trader` model in the finance port).
- So today the engine does **policy evaluation**: the exact distribution of returns of a fixed stochastic policy in a stochastic environment. The integration is offline, through a file. The Demo README's `ms4me.para(env, action, state, N)` is the intended online interface and does not exist yet.
- Library v0 supports a fixed input trace only. An agent that reacts to outputs ("reactive inputs") is future work.

2 Where paratemporal fits in RL

Use	Engine provides	Standard alternative	Why it might win
A. Policy evaluation	exact $V^\pi(s)$ and the full return distribution	many Monte Carlo rollouts; TD	no sampling variance where draws are discrete and merge well
B. n-step targets	$E[G_{t:t+n}]$ from s_t, exact or sampled with merging, leaves bootstrapped from the learner's V	one sampled n-step return; tree backup; $Q(\sigma)$	lower-variance targets; generalises Expected SARSA and tree backup to stochastic environments with a simulator
C. Distributional targets	the distribution of the n-step return	C51, QR-DQN learn it by sampling	exact target distributions; risk measures such as CVaR for finance
D. Lookahead for control	sparse sampling or tree search over the DEVS model, merging turns the tree into a DAG	MCTS, sparse sampling (Kearns, Mansour & Ng 2002)	merging handles transpositions; exactness on small branches

Recommended first target: B (n-step targets), with A as its special case ($n \to$ episode end). It is closest to the proposed method, has a clean ground truth, and plugs into any TD learner.

3 Monte Carlo at every stage, plus merging: variance

Setup. At each stage every live node draws k children of weight w/k. After the stage, nodes in the same state are merged and their weights added. The concern: merging means fewer sample paths are drawn than without merging.

Theory.

- **Unbiased either way.** A child carries w/k, so $E[\sum_i (w/k) f(v_i)] = w\,Ef$. Merging equal continuations and sampling their future afresh preserves this (main_paratemporal, Proposition "Safe merging", sampled case).
- **Why variance rises with a fixed k.** Two nodes of weights w_1, w_2 in the same state, σ^2 the variance of one sampled estimate of that state's future. Unmerged, they have independent futures: $(w_1^2 + w_2^2)\sigma^2/k$. Merged, they share one: $(w_1 + w_2)^2\sigma^2/k$, larger by $2w_1 w_2 \sigma^2/k$. Merging with a fixed k discards independent samples, as suspected, and also saves the work of drawing them.
- **The fix is reallocation.** Keep a per-stage budget B and give merged node i $n_i \approx B\,w_i/W$ children (at least one), each of weight w_i/n_i. Unbiasedness holds because n_i depends only on the past. Since the choice depends on weights, it falls outside the paper's merge proposition and needs its own short argument; dropping nodes ($n_i = 0$) requires Russian roulette to stay unbiased.
- **Safe merging rarely fires in RL.** It requires equal observations, so accumulated returns must match too; with continuous rewards they almost never do. Merging on state only while carrying the probability-weighted accumulated return $A = \sum w\,G_{\text{past}}$ is exact for *expectations*, by linearity ($E[G] = \sum_s A_s + \sum_s W_s\,E[G_{\text{future}} \mid s]$), but loses the return *distribution*. This is the old IH idea with a precise validity condition: a linear objective and an additive reward.

Experiment (`rl/experiments/merge_variance.py`). 19-state random walk, start 13, 6-step target bootstrapped with $V(s) = (s-10)/10$, step reward $s'/100$ so that returns differ between paths, 1000 replications. Work counts transitions sampled.

Estimator	**Mean** (exact 1.08)	**Variance**	**Work**	**Var × work**
exhaustive + accumulator merge	**1.08000 exact**	0	**42**	0
tree, $k = 3$, no merge	1.0796	1.16e-2	1092	12.7
tree, $k = 3$, safe merge	1.0788	1.95e-2	106	2.07
tree, $k = 3$, accumulator merge	1.0827	2.40e-2	51	1.23
budget $B = 9$/stage	1.0775	1.23e-2	54	0.66
budget $B = 182$/stage (= tree work)	1.0799	**6.6e-4**	1092	0.72
plain MC, $N = 9$ rollouts	1.0786	1.29e-2	54	0.70
plain MC, $N = 182$ rollouts	1.0812	5.8e-4	1092	0.63

All estimators are unbiased (each mean within 1.6 standard errors of 1.08).

What this means.

1. A fixed-k tree without merging is the worst use of samples: its k^n leaves share ancestors.
2. Merging with a fixed k is about 20× cheaper and about 2× noisier, a net 6–10× gain.
3. Merging with budget reallocation, at the work of the unmerged tree, gives 17× lower variance and ties with plain independent rollouts per unit of work.
4. With sampling everywhere, merging cannot beat plain Monte Carlo. The engine's advantage is **exhaustive expansion with merging wherever outcome sets are small**. The design for n-step methods is therefore **hybrid**: enumerate small discrete draws, merge, and sample only large or continuous draws with budget reallocation.

Open questions: allocation by weight or by weight × spread (Neyman allocation); effective sample size per stage; repeating the study with larger branching and continuous noise.

4 Toy problems

#	Problem	Why	Ground truth
T1	19-state random walk (S&B §7.1)	the standard n-step TD benchmark (S&B Fig. 7.2)	$V(s) = (s-10)/10$
T2	FrozenLake 4×4 / 8×8, slippery (Gymnasium)	stochastic transitions, small state space so merging is very effective; control	value iteration
T3	Blackjack (Gymnasium)	classic Monte Carlo control, discrete rewards (safe merging works)	known optimal policy
T4	Admission control / (s, S) inventory with random event times (DEVS-native)	real event times, semi-MDP: where DEVS beats step-based gyms	DP on the embedded chain
T5	Finance single stock (existing port)	ties back to the Demo Deliverable; risk measures	engine exhaustive results

Order: T1 → T2 → T5 → T4; T3 optional.

5 Python baselines

Framework	Use
Gymnasium (Farama)	environment API; FrozenLake, Blackjack, CliffWalking built in; our DEVS environments get a Gymnasium wrapper
Own tabular reference code	n-step TD, n-step SARSA, tree backup, $Q(\sigma)$ (S&B ch. 7); transparent and the fairest baseline for T1/T2
Stable-Baselines3 + sb3-contrib	PPO, DQN, A2C; QR-DQN for distributional comparison
CleanRL	single-file implementations; easiest place to swap in engine-computed n-step targets
optional: MCTS baseline	own UCT, or DeepMind's JAX-based `mctx`, for use D
not now: RLlib, TorchRL, Tianshou	heavier; revisit if scale demands it

6 Connecting the Java/MS4 engine to Python

Two directions (`Paper/DEVS-RL/DEVS-RL_Integration.png`). *DEVS into RL*: the DEVS model is the environment and a Python agent calls `reset/step`; the engine must pause at a decision point and wait for an action, or for the policy's action probabilities to branch over. *RL into DEVS*: the agent is a DEVS component (like `Trader`) whose policy is a table or, later, an exported network (ONNX inside Java). The second works today for tabular policies.

A unifying library feature: decision points. Add `Para.decide(label, options)`, analogous to `Para.choose`, whose law comes from a *policy supplied at run time* (a table, or a callback into Python). The engine treats it like any draw (exhaustive, sampled or hybrid), so one mechanism covers policy evaluation, n-step targets and branching over actions. A Gymnasium `step(action)` is the special case of a policy that forces one action.

Transport options.

Option	How	Fit
1. Files (today)	policy CSV → Java → results	offline tabular evaluation only
2. JPype (in-process)	the JVM runs inside Python (JNI); Python calls `ParaEngine` directly and can implement Java interfaces	**recommended first**: lowest latency, no server; verified working (Section 8)
2b. Py4J	a separate JVM reached over a local socket	simpler isolation, more overhead per call
3. Service	Java engine behind gRPC, REST or ZeroMQ	learners and simulators on different machines
4. Message broker	MQTT, Kafka or RabbitMQ between DEVS federates and agents	distributed, real-time or multi-framework co-simulation

Recommendation: one small Python API (`ParaEnv` for Gymnasium plus `para_eval(state, policy, n, rule)`), implemented first over JPype and kept independent of the transport.

The "online DEVS message broker".

- **MS4 Me's built-in collaboration service is not a broker.** Inspected locally in the MS4 Me 1.5 plugins: a SOAP client for login, licences and the MS4 model store (upload, download and lock model jars).
- **Closest match:** Dominici, Capocchi, Santucci, Koertje, Kewley, *Broker-Based Distributed DEVS Simulation: Enabling Multi-Framework Interoperability through Message Broker Architecture*, SIMULTECH 2026, doi:10.5220/0014846100004094. Capocchi and Santucci created DEVSimPy (Python DEVS), so this is likely the Python-connectable broker remembered. Its broker technology and code availability could not be confirmed online. **To do: obtain the paper.**
- Related: xDEVS (Java, Python, C++; distributed over sockets and, in 2025 work, MQTT); DEVS/SOA (web services).

- Assessment: a broker matters for real-time or multi-framework co-simulation over a network. For RL training, where the learner calls the engine very often, an in-process bridge is the better first step.

7 Phased plan

Phase	Work	Done when
R0 setup	project virtual environment; pinned gymnasium, stable-baselines3, jpype1; notebook run headless	notebook Q-table reproduced locally
R1 Python prototypes	reference estimators; n-step TD on T1 with single-path, exact, sampled+merge+budget and rollout targets; then T2	learning curves showing where model-based targets win per unit of work
R2 library features	k-sample Monte Carlo rule; per-stage budget; accumulator merge (*needs approval*); leaf-value hook; `Para.decide`	new Java acceptance tests pass
R3 bridge	JPype `ParaEnv` and `para_eval`; T2 through the Java engine	Java matches the Python prototype
R4 baselines	T2/T5 (then T4) vs tabular code and SB3	comparison report
R5 architecture	read Dominici et al.; decide on a service or broker	decision recorded

Note. Decisions (updated 2026-10-16). (1) Accumulator (expectation-only) merging: **approved as an opt-in** for the Java library, off by default; in progress. (2) JPype-first integration: being followed. (3) Toy-problem order T1 $\rightarrow$ T2 $\rightarrow$ T5 $\rightarrow$ T4: being followed. (4) Control: tabular first, the learner in Python, online action selection first, one RL step = one decision, discounting by simulation time (γ^{τ}).

8 Progress

8.1 R0: environment (done 2026-10-10)

- Virtual environment `rl/.venv` (Python 3.13, system site-packages for numpy). Pinned in `rl/requirements.txt`: gymnasium 1.4.0, stable-baselines3 2.9.0, sb3-contrib 2.9.0, jpype1 1.7.1, torch 2.14.1 (inside the venv; the system torch 2.7 is untouched), matplotlib 3.11.2, pandas 2.3.3, numpy 2.3.4.
- **JPype bridge verified**: `rl/scratch/jpype_smoke.py` starts the JVM from Python, implements the Java `Transducer` and `Supplier` interfaces in Python, runs the tutorial `ServiceLine` through the Java `ParaEngine`, and gets the exact result ($P(3 \text{ served}) = 0.36$, $P(2) = 0.64$).
- **Q-table notebook runs headless**: `rl/baselines/q_table_constructor.py` ports the notebook's training cells (numpy instead of torch, a seed, no Colab). Trained on `Train_AAPL.csv` with 15 shares it reproduces the Demo's `Q_table_AAPL_stoch_1_15.csv` to within about 1% per entry (buy probability in row 0: 0.792 vs 0.795; the difference is the random seed). Runs in about 7 s.

Note. Finding about the existing RL reward. The notebook's reward is 1 when the new holding equals the reachable holding of largest value $\sum$shares $\times$ price at the current date. With positive prices that is always the largest share count, so the reward means "buy" (or hold at the cap), regardless of price movements. The learned policies are therefore buy-biased by construction (about 80% buy in every row) and carry no information about price dynamics. For T5 the reward should be redesigned, for example as the change in portfolio value, before policies from it are interpreted as trading strategies.

8.2 R1: Python reference prototypes (in progress)

`rl/paratemporal_py` implements tabular Markov reward processes and four n-step target estimators: exact (exhaustive expansion with merging), budgeted sampling with merging, independent rollouts, and the classic trajectory return. Under expectation merging the merged nodes at a stage form a weight vector over states, so the estimators are vectorised. `rl/tests/test_targets.py` checks Bellman consistency (with the true V, every exact n-step target equals $V(s)$, to 10^{-12}), agreement with matrix powers, and unbiasedness of the sampled estimators. All pass. `paratemporal_py.frozen_lake` loads the exact slippery-FrozenLake model from Gymnasium; its exact 100-step success probability under the optimal policy (0.7402) matches Gymnasium's own Monte Carlo (0.741 ± 0.003).

Done: prediction studies on T1 and T2 (below), repeated in the Java engine with k-branch sampling (Section 8.6). Remaining in R1: T2 control, hybrid selection, and the work-saturation study (both in progress).

Setup. Each study compares four ways of forming the target for updating $V(S_t)$. (A) Classic n-step TD uses only the agent's own trajectory (model-free). (B) The exact expected n-step target is computed by exhaustive expansion with merging. (C) The sampled target uses merging and a per-stage budget of 4. (D) The target averages 4 independent rollouts. B, C and D query a simulator from every visited state, which is the paratemporal setting. Two costs therefore matter separately: *real experience* (episodes) and *compute* (work, the transitions examined or sampled). Both runs were restarted in the foreground with incremental saving after an out-of-memory kill.

8.3 T1: 19-state random walk, n-step TD prediction

Target	**n=1**	**n=2**	**n=4**	**n=8**
A: n-step TD (model-free)	0.345 ± 0.002	0.276 ± 0.004	0.263 ± 0.005	0.277 ± 0.005
best α, work/episode	α=0.8, 99	α=0.6, 99	α=0.4, 99	α=0.2, 99
B: exact (exhaustive + merge)	0.285 ± 0.005	0.238 ± 0.006	0.181 ± 0.006	0.139 ± 0.006
best α, work/episode	α=1, 296	α=1, 687	α=1, 2,030	α=1, 6,693
C: sampled + merge + budget	0.305 ± 0.006	0.279 ± 0.005	0.239 ± 0.005	0.217 ± 0.004
best α, work/episode	α=1, 495	α=1, 886	α=0.8, 1,711	α=0.6, 3,268
D: independent rollouts	0.305 ± 0.006	0.280 ± 0.005	0.242 ± 0.005	0.218 ± 0.005
best α, work/episode	α=1, 495	α=1, 913	α=0.8, 1,632	α=0.6, 3,231

Best step size per cell. RMS error over the 19 states, averaged over the first 10 episodes and 100 runs. Work counts transitions examined or sampled, including the agent's own steps.

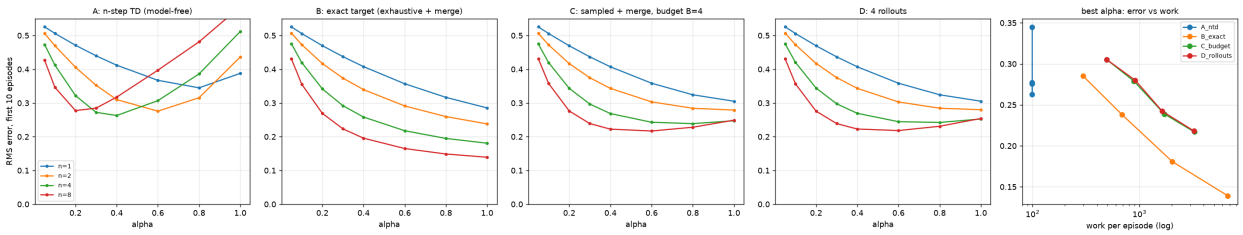


Figure 1: T1: 19-state random walk, n-step TD prediction: error against step size for each target, and best-α error against work.

8.4 T2: slippery FrozenLake 8×8, prediction

Target	**n=1**	**n=2**	**n=4**	**n=8**
A: n-step TD (model-free)	0.185 ± 0.002	0.174 ± 0.002	0.166 ± 0.002	0.161 ± 0.002
best α, work/episode	α=0.6, 42	α=0.4, 42	α=0.4, 42	α=0.2, 42
B: exact (exhaustive + merge)	0.159 ± 0.002	0.137 ± 0.002	0.114 ± 0.003	0.094 ± 0.003
best α, work/episode	α=1, 205	α=1, 804	α=1, 4,067	α=1, 21,502
C: sampled + merge + budget	0.168 ± 0.002	0.147 ± 0.003	0.135 ± 0.003	0.124 ± 0.002
best α, work/episode	α=1, 201	α=1, 380	α=0.8, 676	α=0.6, 1,358
D: independent rollouts	0.168 ± 0.002	0.150 ± 0.003	0.135 ± 0.003	0.123 ± 0.002
best α, work/episode	α=1, 201	α=1, 359	α=0.8, 694	α=0.6, 1,227

Best step size per cell. RMS error over nonterminal states, averaged over the first 30 episodes and 50 runs (exploring starts; $\varepsilon = 0.2$-soft optimal policy; $\gamma = 0.99$). Work counts transitions examined or sampled, including the agent's own steps.

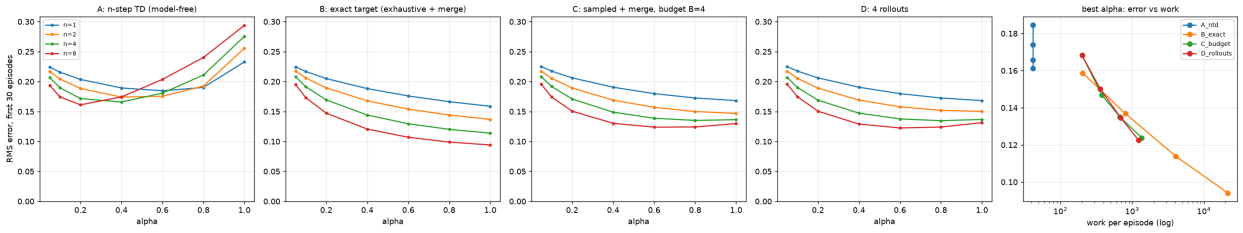


Figure 2: T2: slippery FrozenLake 8×8, prediction: error against step size for each target, and best-α error against work.

8.5 What R1 shows

1. **The baseline is reproduced.** A reproduces Sutton & Barto's Figure 7.2 on T1: intermediate n is best ($n = 4$, RMS 0.263 at $\alpha = 0.4$), with the characteristic U-shaped curves in α.
2. **Per episode of real experience, the exact target wins clearly.** On T1, B with $n = 8$ reaches RMS 0.139 against 0.263 for the best model-free setting, roughly half the error after the same 10 episodes. On T2 it is 0.094 against 0.161. B's error falls steadily with n and is

best at $\alpha = 1$ in every cell: the exact target has no sampling noise, so there is no reason to average it.

3. **Merging with a budget ties with independent rollouts, again.** C and D agree within one standard error in every cell of both studies (e.g. T1, $n = 8$: 0.217 vs 0.218; T2, $n = 8$: 0.124 vs 0.123). This matches the variance study in Section 3: for sampled expected-value targets, merging with reallocation is as good as, but not better than, independent sampling.
4. **Per unit of compute, the picture depends on the problem.** On T1 the exact target dominates the sampled ones at equal work (right panel: B's curve lies below C and D everywhere; e.g. 0.181 at 2,030 work vs 0.217 at about 3,250). On T2 the three are about level at equal work (e.g. B $n = 2$: 0.137 at 804; C/D $n = 4$: 0.135 at about 690). B buys lower error at higher cost, because an ε-soft policy on slippery ice has up to 12 successors per state and the merged frontier quickly covers much of the grid.
5. **Against model-free TD, the comparison is about which resource is scarce.** B–D spend 3–500× more work per episode than A. If real experience is expensive and a simulator is available (the DEVS setting), the paratemporal targets are much better. If compute and experience cost the same, model-free TD with more episodes remains competitive on these small problems.

Consequences for the next phases.

- The case for exact expansion with merging is strongest when the *reachable merged frontier stays small*: work per target is bounded by $n \times |\text{frontier}| \times$ branching, independent of the number of paths, while a sampled estimate needs ever more samples for the same accuracy. Next: measure where exact expansion's work saturates as n grows, and how many samples C/D need to match B's error.
- Hybrid selection (exact on small discrete draws, sampled elsewhere) is the configuration to test next. On T2 the natural split is to enumerate the slip (3 outcomes) and sample the policy, or the reverse.
- Control (T2 with n-step Q-learning or SARSA targets) and the Java engine via JPype (R3) are the next steps. The Python estimators above are the reference the Java results must match.

8.6 R1 in the Java engine: n-step targets with k branches (2026-10-14)

Note. In short. The same question, asked of the real engine rather than the Python prototype: does the paratemporal engine help n-step Monte Carlo with k branches per node? Yes. Merging makes a k-tree 5–10× more efficient, and **budget reallocation with merging is the best sampled method in all 12 settings**, 1.2–2× more efficient than independent rollouts. Exhaustive expansion with merging gives the exact target at a cost close to a single sampled estimate on the random walk, and 7× cheaper than unmerged exhaustive expansion on FrozenLake.

Setup. `examples.rl.MdpWalker` follows a fixed policy in a tabular MDP and draws its next state when it *leaves* a state, after the engine has merged the branches that reached it ("merge, then sample"). The n-step expected target $E[\sum_{i \le n} \gamma^{i-1} R_i + \gamma^n V(S_n) \mid S_0 = s]$ is computed with $V = V^\pi$, so every exact target equals $V^\pi(s)$ and bias is measured against a known value. Problems: the

19-state random walk (5 start states) and slippery FrozenLake 8×8 with the ε-soft optimal policy ($\varepsilon = 0.2$, $\gamma = 0.99$; 6 start states); $n = 2, 4, 6$; $k = 2, 3$; 200 replications per sampled method. Two new selection rules make this possible: `SelectionRule.monteCarlo(k)` (k samples per draw, weight w/k each) and `SelectionRule.budget(B)` ($\max(1, \text{round}(Bw))$ samples per draw for a branch of weight w). Work is the engine's departure count; efficiency is variance $\times$ work (lower is better; for an unbiased estimator variance falls as 1/work).

	Random walk, $n = 6$, $k = 3$		**FrozenLake** 8×8, $n = 6$, $k = 3$	
Method	**Work**	**Var $\times$ work**	**Work**	**Var $\times$ work**
k-tree, no merge	2,132	10.9	1,812	0.616
k-tree + merge	102	1.02	213	0.122
budget $B = 9$ + merge	82	**0.546**	85	**0.042**
9 independent rollouts	125	0.835	115	0.053
exhaustive, no merge	249	exact	5,060	exact
exhaustive + merge	102	exact	730	exact

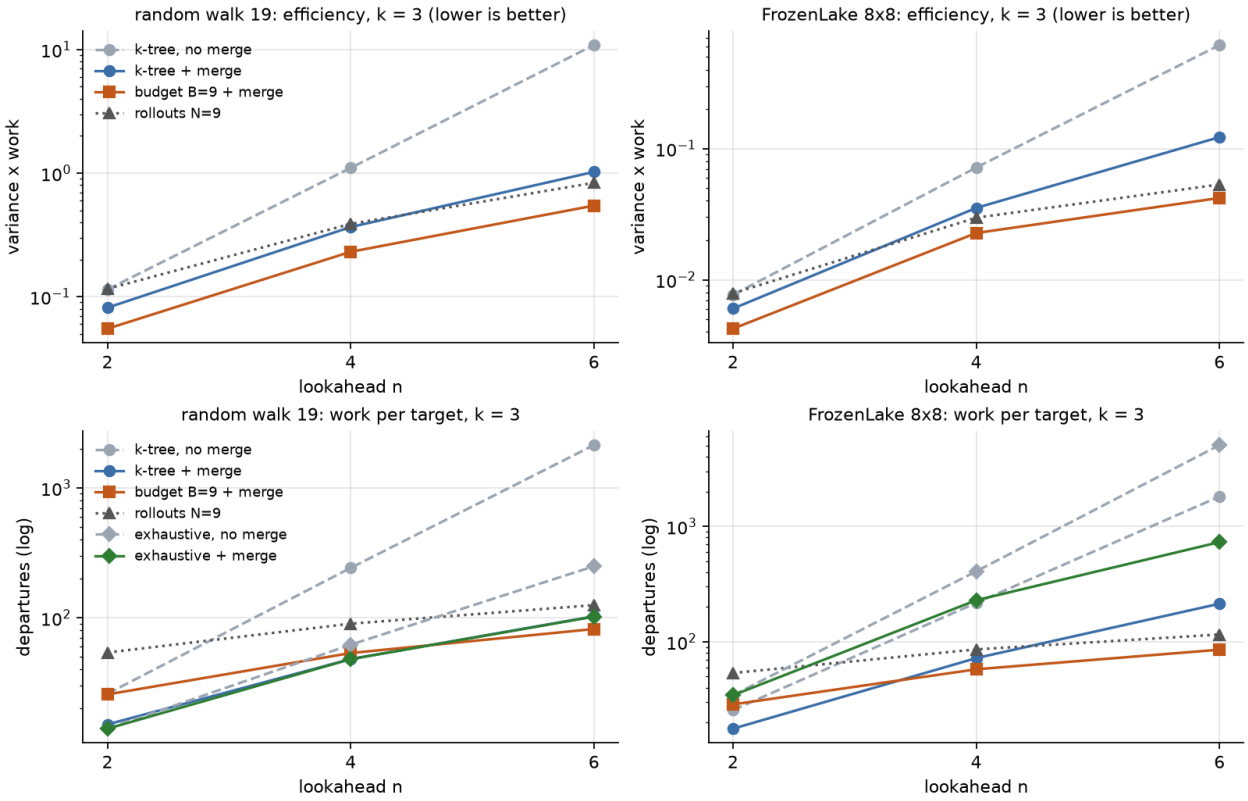


Figure 3: Java engine, $k = 3$: efficiency (top) and work per target (bottom) against lookahead n. Full results for $k = 2, 3$ and $n = 2, 4, 6$ are in `rl/experiments/results/java_nstep_compare.csv`.

What it shows.

1. **All sampled estimators are unbiased**: every mean is within about 2 standard errors of the exact target.

2. **Merging a k-tree pays**: work falls 8–21× at $n = 6$, $k = 3$, variance rises only 1.5–2× (merged branches share one sampled future, as Section 3 predicts), so efficiency improves 5–10×. The gain grows with n and k; at $n = 2$ there is little to merge.
3. **A merged k-tree with a fixed k is still only level with independent rollouts**, and worse on FrozenLake at $n = 6$. The fix is the one the variance analysis gives: **reallocate the budget by weight**. Budget + merge is the most efficient sampled method in every one of the 12 settings, 1.2–2× better than rollouts. Part of that edge is that the tree simulates the shared first steps once, where each rollout repeats them.
4. **Exact expansion is competitive.** On the random walk, exhaustive + merge costs about one sampled estimate and has no variance. On FrozenLake it costs 730 departures; at budget + merge's efficiency, any standard deviation below about $\sqrt{0.042/730} \approx 0.008$ costs more than that by sampling, so exact wins whenever that precision is needed.

Found and fixed along the way.

- **Correlated seeds.** `java.util.Random` gives nearly equal first values for nearby seeds: across seeds $7, 8, 9, \ldots$ the first draw from a 4-outcome law never produced one of the outcomes. Averages over replications seeded $1, 2, 3, \ldots$ were therefore biased (all sampled estimators came out low, by up to 9 s.e.). The engine now passes the seed through a SplitMix64 scramble, so nearby seeds give independent streams.
- **FrozenLake ground truth.** $V^*(\text{start}) = 0.414640361787$ at $\gamma = 0.99$; the Java model and the Gymnasium-derived Python model agree to 10^{-11} (an earlier note of 0.542 was wrong).
- **Results file.** Method names containing commas were written unquoted; the CSV writer now quotes text.

Checks. `examples.rl.VerifyNStep` (22 checks, part of the build): exact targets equal $V^\pi(s)$ to 10^{-12} with and without merging and with one or two draws per step, V^* matches the Python value, merging reduces work, and the four sampled estimators are unbiased within 4 s.e. `CompareNStep` runs its replications in parallel (`-Dthreads`); results are identical for any thread count.

Caveats. Both problems pay rewards only on arrival in end states, so safe merging fires; with a reward on every step it needs the expectation-only merge (approved as an opt-in, in progress). These are prediction results for a fixed policy; control is the next phase.